

Towards partial fulfillment for Postgraduation Degree level
Program

Master of Technology in Computer Science and Engineering
Specialization with Information Security and Privacy

A Mini-Project Report on:

NIDS-FGPA: Federated Learning Network Intrusion Detection with GSA and Homomorphic Encryption

Prepared by:

Jivani Aayush

Roll No: P25IS024

Class : M.TECH I(Computer Science and Engineering) 2nd
Semester

Year : 2025-2026

Supervisor: Dr. Devesh C. Jinwala



Department of Computer Science and Engineering
Sardar Vallabhbhai National Institute of Technology, Surat

May 5, 2026

Student Declaration

This is to certify that the work described in this mini project report has been actually carried out and implemented by Jivani Aayush : P25IS024 . Neither the source code therein nor the content of the project report have been copied or downloaded from any other source. I understand that my graded result would be revoked if it is later found to be so.

Jivani Aayush
Roll No: P25IS024

Certificate

This is to certify that the mini project report entitled “NIDS-FGPA: Federated Learning Network Intrusion Detection with Gradient Similarity Aggregation and Homomorphic Encryption” is prepared and presented by Jivani Aayush (Admission No.: P25IS024) of MTech First Year in Computer Science and Engineering with Specialization in Information Security and Privacy and her work is satisfactory.

Signature:

Supervisor/s

Jury

Head of Department

Acknowledgment

I would like to express my sincere gratitude to my supervisor for their continuous guidance, encouragement, and support throughout the course of this mini-project. I am also thankful to the Department of Computer Science and Engineering, Sardar Vallabhbhai National Institute of Technology, Surat, for providing the necessary computational resources and infrastructure.

I extend my gratitude to the authors of the primary reference paper, Wang et al. (2024), whose work on NIDS-FGPA provided the foundational framework for this study. I also acknowledge the creators of the Edge-IIoTset and CIC IoT 2023 datasets for making their data publicly available for research.

Finally, I thank my peers and family for their constant moral support and encouragement.

Abstract

As more IIoT systems are adopted in various sectors including manufacturing, healthcare, and smart grid, the security of such network infrastructures becomes essential. In this context, classical IDSs employ a central authority that collects all data samples, thereby posing a privacy threat and sometimes being unreasonable owing to regulations and other restrictions. Federated learning allows multiple parties to collaborate while keeping data confidential and can cope with Non-IID dataset distribution and gradient inversion attacks that might lead to information leakage in traditional methods such as FedAvg.

proposed federated intrusion detection solution NIDS-FGPA consists of four key components: (1) a pre-processing technique that converts network flows' features into 8x8 grayscale images for spatial learning, (2) a 2DCNN-BiGRU model to learn spatial and temporal information from the images, (3) a Gradient Similarity Aggregation (GSA) algorithm that performs filtering and weighting based on cosine similarity between gradients and (4) Paillier encryption. Under extreme Non-IID conditions, experiments on Edge-IIoTset and CIC IoT 2023 datasets demonstrate GSA obtains an accuracy of 0.9558 with only 19.4 MB communication overhead, which outperforms existing algorithms (FedAvg, FedProx, FedNova).

Keywords: Federated Learning, IIoT Security, Intrusion Detection System, Non-IID Data, Gradient Similarity Aggregation, Homomorphic Encryption

Contents

List of Figures	vii
List of Tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Objectives	1
1.3 Report Organisation	2
2 NIDS-FGPA: System Overview	3
2.1 Proposed Solution	3
2.2 System Architecture	3
3 Methodology: Four Core Algorithms	4
3.1 Algorithm A: Traffic to Grayscale Image Pipeline	4
3.1.1 Pipeline Steps	4
3.1.2 Why Images?	4
3.2 Algorithm B: 2DCNN-BiGRU Model Architecture	5
3.2.1 Layer Stack	5
3.2.2 BiGRU Gate Equations	5
3.2.3 Training Configuration	5
3.3 Algorithm C: Gradient Similarity Aggregation (GSA)	5
3.3.1 Client-Side: Local Training and Filtering	6
3.3.2 Server-Side: Dynamic Weighted Aggregation	6
3.4 Algorithm D: Paillier Homomorphic Encryption	6
3.4.1 Key Generation	6
3.4.2 Encrypt and Decrypt	7
3.4.3 Homomorphic Properties	7
4 Datasets and Experimental Setup	9
4.1 Datasets	9
4.1.1 Edge-IIoTset	9
4.1.2 CIC IoT 2023 (UNB)	9
4.2 Non-IID Client Configuration	9
4.3 Baseline Algorithms	9
4.4 Evaluation Metrics	10
4.5 Hyperparameter Configuration	10
5 Results and Analysis	12
5.1 Convergence Analysis	12
5.2 Performance and Communication Overhead	12
5.3 Convergence Speed	13
5.4 Per-Class Analysis	13
5.5 GSA Filter Analysis	14

6	Design Choices, Pitfalls, and Limitations	18
6.1	Design Choice Justification	18
6.2	Implementation Pitfalls	18
6.2.1	Algorithm-Level Pitfalls	18
6.2.2	Engineering Pitfalls	18
6.3	Limitations	19
7	Conclusion and Future Work	20
7.1	Conclusion	20
7.2	Future Directions	20
	Bibliography	21
A	Key Code Listing	22
A.1	Pseudocode: Normal Federated Averaging (FedAvg)	22
A.2	Pseudocode: GSA – Gradient-Similarity-Weighted Aggregation	22
A.3	Pseudocode: Network Traffic to 8×8 Grayscale Image	24
A.4	Pseudocode: 2D CNN-BiGRU Model Architecture	24
A.5	Pseudocode: Client Data Partitioning (IID and Non-IID)	25

List of Figures

2.1	NIDS-FGPA Three-Tier Federated Pipeline	3
3.1	Sample 8×8 Grayscale Traffic Images per Attack Class. Each class produces distinct pixel patterns, enabling 2D spatial learning by the CNN.	4
3.2	Paillier Homomorphic Encryption Demo. Top-left: lossless encrypt–decrypt cycle. Top-centre: additive HE verification. Top-right: scalar multiplication HE verification. Bottom-left: weighted aggregation on ciphertext matches plaintext result. Bottom-centre: ciphertext size vs. key length (1024-bit chosen). Bottom-right: full encryption workflow (Algorithm 2).	8
4.1	Client Data Distribution (C=3, Non-IID Setup). Left: <code>client_noniid_0</code> holds only 320 MITM samples. Centre and right: two IID clients each hold 887,194 samples across all 6 attack classes.	10
5.1	Convergence of GSA vs All Baselines (C=3). Left: accuracy per round — GSA and FedProx converge above the paper target (0.945, red dashed line) by Round 5; FedNova collapses at Round 20. Right: loss per round — FedNova loss spikes to 3×10^{10} while all other methods converge smoothly.	12
5.2	GSA Training Convergence: C=3 vs C=6. GSA C=3 consistently exceeds the paper target accuracy (0.945); GSA C=6 converges more slowly due to greater client heterogeneity.	12
5.3	Performance and Communication Overhead Comparison (Paper Table 6 style). Left: accuracy — GSA C=3 matches paper benchmark (red star = 0.945). Centre: weighted F1 scores. Right: communication overhead in MB — GSA C=3 achieves 19.4 MB vs paper’s 45.6 MB; FedNova fails on accuracy.	13
5.4	Communication Overhead: GSA C=3 (19.4 MB) vs GSA C=6 (42.9 MB). FedAvg and FedProx overhead values are not shown as they are equal to GSA C=3 (20.1 MB).	14
5.5	Model Performance Summary vs Paper Benchmark. Our GSA C=3 outperforms the paper on all four metrics (Accuracy: 0.9558 vs 0.9450; Precision: 0.9615 vs 0.9400).	14
5.6	Rounds to First Reach 0.93 Accuracy. FedNova (green bar, labelled “Never”) never reaches the 0.93 threshold. FedProx achieves it at R1; GSA and FedAvg at R5; GSA C=6 at R15.	15
5.7	Per-Class F1 Heatmap: All Methods vs All Attack Classes. Green = high F1; red = low/zero. MITM F1 = 0 for all methods. FedNova fails on Malware and Scanning entirely. GSA C=3 and FedProx are the strongest across minority classes.	15
5.8	Per-Class F1 Score: GSA C=3 vs C=6. Both configurations achieve perfect F1 on Normal and DDoS. MITM F1 = 0 for both due to class imbalance. GSA C=3 outperforms C=6 on Injection (0.73 vs 0.64) and Scanning (0.72 vs 0.00). . . .	16
5.9	Confusion Matrices: GSA C=3 (left) vs GSA C=6 (right). C=3: Normal and DDoS are perfect; Injection 85%; MITM 0% (all predicted as DDoS due to class imbalance); Malware 40%; Scanning 60%. C=6: MITM improves dramatically to 96% recall; Injection improves to 90%.	16

5.10	GSA Gradient Filter Analysis (C=3). Left: upload vs blocked clients per round — Non-IID client is blocked from Round 15 onward. Centre: block rate (pink bars, left axis) vs accuracy (blue line, right axis) — 33% block rate with stable 0.955 accuracy. Right: cumulative communication overhead — GSA (blue) saves bandwidth vs FedAvg (orange dashed) with savings shown as green shaded area.	17
5.11	GSA Client Filtering Behaviour (C=3). Left: uploaded (green) vs blocked (red) clients per evaluation round. Right: Non-IID client block rate over training — stabilises at the 1/3 dotted reference line from Round 15 onward.	17
5.12	Complete Results vs Paper Benchmarks (yellow rows = paper values; green row = our best result). Our GSA C=3 exceeds paper accuracy (0.9558 vs 0.9450) and reduces overhead from 45.6 MB to 19.4 MB. FedProx C=3 achieves the fastest convergence (R1). FedNova fails under the strict Non-IID setup (0.8946 vs paper's 0.9260).	17

List of Tables

1.1	Existing Approaches and Their Limitations	1
4.1	Non-IID Client Data Distribution (C=3)	9
4.2	Experimental Hyperparameters	11
5.1	Performance Comparison: Our Implementation vs. Paper Benchmarks	13
5.2	Rounds to First Reach 0.93 Accuracy	15
6.1	Design Choice Justifications	18
7.1	Problem-Solution Mapping in NIDS-FGPA	20

List of Abbreviations

Abbreviation	Full Form
2DCNN	Two-Dimensional Convolutional Neural Network
BiGRU	Bidirectional Gated Recurrent Unit
CIC	Canadian Institute for Cybersecurity
DDoS	Distributed Denial of Service
DoS	Denial of Service
ECG	Electrocardiogram
Edge-IIoTset	Edge Industrial Internet of Things Dataset
F1	F1-Score (Harmonic Mean of Precision and Recall)
FedAvg	Federated Averaging
FedNova	Federated Nova (Normalized Local Gradient Aggregation)
FedProx	Federated Proximal
FL	Federated Learning
GSA	Gradient Similarity Aggregation
GRU	Gated Recurrent Unit
HE	Homomorphic Encryption
IDS	Intrusion Detection System
IIoT	Industrial Internet of Things
IoT	Internet of Things
IID	Independent and Identically Distributed
MITM	Man-in-the-Middle
MPC	Multiparty Computation
NIDS	Network Intrusion Detection System
NIDS-FGPA	Network IDS with Federated learning, Gradient similarity, Paillier, and Aggregation
Non-IID	Non-Independent and Identically Distributed
Recon	Reconnaissance
ReLU	Rectified Linear Unit
SMOTE	Synthetic Minority Over-sampling Technique
TP	True Positive
TN	True Negative
UNB	University of New Brunswick

1. Introduction

1.1 Motivation and Problem Statement

The IIoT encompasses various critical domains like factories, healthcare facilities, and smart grids, among others, that involve highly confidential traffic over networks. The current IDS architecture works on a centralized model, where there will be an inevitable requirement of sharing raw data, thereby exposing organizations to privacy risks through the process. Also, the current IDS architectures cannot be scalable for heterogeneous edge nodes present in IIoT systems.

The attack surfaces in an IIoT system include DDoS attack, injection attacks, reconnaissance, MITM, and malware. There are three basic hard constraints that pose a challenge to the problem:

1. Organisations individually hold **small and incomplete** local datasets.
2. Data owners **refuse** to share raw network traffic due to legal, competitive, or privacy reasons.
3. Naive model aggregation in federated settings is **vulnerable** to gradient poisoning from malicious or low-quality clients.

Table 1.1: Existing Approaches and Their Limitations

Approach	Limitation
Centralised IDS	Privacy leakage; single point of failure
Standard FL (FedAvg) [2]	Non-IID data causes diverging gradients; equal weighting ignores update quality
FedNova [4]	Gradient instability on heterogeneous data; training collapse under extreme Non-IID
Differential Privacy	Adds calibrated noise resulting in accuracy degradation
FL without encryption	Server reconstructs data via gradient inversion attacks

The Gap: No existing FL-IDS method jointly addresses Non-IID filtering, gradient privacy, and model accuracy simultaneously.

1.2 Objectives

The main goals of this mini-project include the following:

1. Analysis, understanding, and implementation of the NIDS-FGPA scheme introduced by Wang et al. (2024) [1].

2. Implementation of three federated learning approaches as baselines (FedAvg, FedProx, and FedNova).
3. Comparison of all techniques with respect to the practical datasets (Non-IID datasets Edge-IIoTset and CIC IoT 2023).
4. Trade-off analysis of convergence time vs. communication cost vs. classification accuracy.
5. Discussion of challenges of implementation and rationale behind decisions.

1.3 Report Organisation

The remainder of this report is structured as follows. Chapter 2 provides an overview of the proposed NIDS-FGPA system. Chapter 3 details the four core algorithmic components. Chapter 4 describes the datasets, experimental setup, and evaluation metrics. Chapter 5 presents and analyses experimental results. Chapter 6 discusses design choices, pitfalls, and limitations. Chapter 7 concludes with future directions.

2. NIDS-FGPA: System Overview

2.1 Proposed Solution

NIDS-FGPA is a federated learning-based intrusion detection system algorithm consisting of four closely related parts:

1. **1. Data Layer:** Grayscale 8×8 images are made out of raw features of the network traffic.
2. **2. Model Layer:** A hybrid 2DCNN-BiGRU neural network carries out classification.
3. **3. Aggregation Layer:** Dynamic weighted model aggregation is ensured by the Gradient Similarity Aggregation (GSA) algorithm.
4. **4. Privacy Layer:** All gradient uploads are encrypted using the Paillier Homomorphic Encryption technique.

The key insight is that gradient *direction* (measured via cosine similarity) reliably identifies harmful client updates before aggregation. Combined with encryption, the central server never sees plaintext gradient values.

2.2 System Architecture

The entire process is based on a three-tiered federated pipeline system depicted below. The IIoT edge clients do the local training, then GSA filtering, Paillier encryption, homomorphic summation in the server side, and decryption of the weight updates by the client itself.

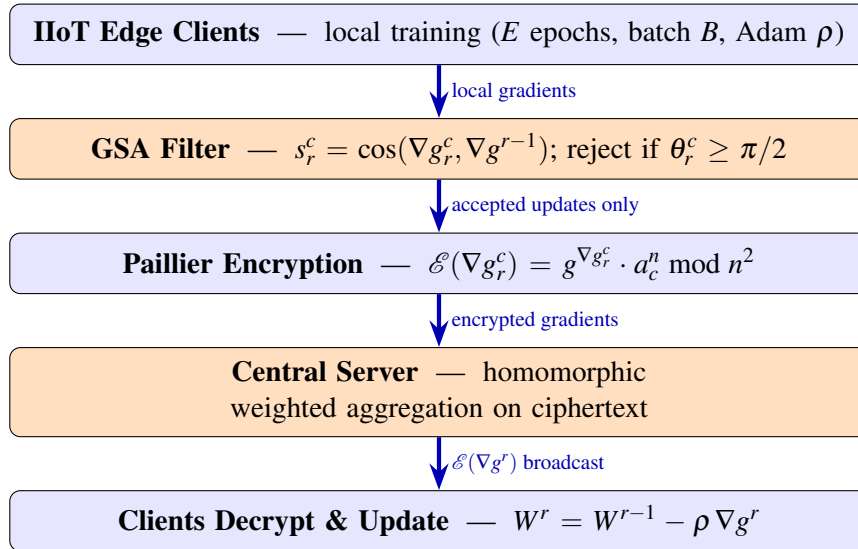


Figure 2.1: NIDS-FGPA Three-Tier Federated Pipeline

Privacy properties: Clients do not share the raw dataset; Server does not see the plain-text gradients; Divergent clients are dropped silently.

3. Methodology: Four Core Algorithms

3.1 Algorithm A: Traffic to Grayscale Image Pipeline

However, raw network flow data cannot be inputted into a 2D Convolutional Neural Network. The preprocessing step converts tabular flow data into 2D images, making it possible for the network to take advantage of 2D spatial correlation among the features.

3.1.1 Pipeline Steps

1. **Cleaning:** Remove rows containing NaN values, $\pm\infty$, and duplicate records to ensure data quality.
2. **Normalisation:** Apply min-max scaling to map all features to $[0, 255]$:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

3. **String encoding:** Convert categorical string columns to their ASCII character sum, scaled proportionally to $[0, 255]$.
4. **Zero-padding:** Pad the feature vector to the next perfect square dimension:
 - Edge-IIoTset: 55 features $\rightarrow$ 64 dimensions (8×8 image)
 - CIC IoT 2023: 83 features $\rightarrow$ 81 dimensions (9×9 image)
5. **Reshape:** Convert the 1D padded vector into an $H \times W$ pixel matrix.
6. **Label attachment:** Attach integer class label ℓ to produce a labelled $H \times W \times 1$ grayscale image.

3.1.2 Why Images?

Neighbourhood relationships of features become 2D learnable patterns through the image representation form that would not be possible in a 1D vector representation. Missing features are easily handled by zero-padding while other feature values remain unaffected.

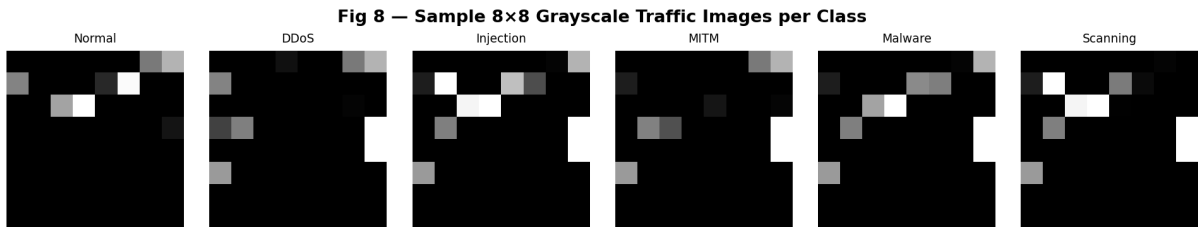


Figure 3.1: Sample 8×8 Grayscale Traffic Images per Attack Class. Each class produces distinct pixel patterns, enabling 2D spatial learning by the CNN.

3.2 Algorithm B: 2DCNN-BiGRU Model Architecture

At each client site, the local model comprises a 2D CNN for extracting spatial features and a BiGRU for modelling sequence/temporal features.

3.2.1 Layer Stack

1. Conv2D(32 filters, 3×3) + ReLU
2. Conv2D(64 filters, 3×3) + ReLU
3. MaxPool2D(2×2 , stride 1)
4. Conv2D(128 filters, 3×3) + ReLU
5. Conv2D(128 filters, 3×3) + ReLU
6. MaxPool2D(2×2 , stride 1)
7. Reshape m_2 as sequence $\rightarrow$ **BiGRU**(128 units)
8. Dropout(0.5)
9. Fully Connected $\rightarrow$ Softmax $\rightarrow \hat{y}$

3.2.2 BiGRU Gate Equations

The BiGRU controls information flow via update gate z_t and reset gate r_t :

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad (3.1)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad (3.2)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad (3.3)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (3.4)$$

The bidirectional structure concatenates forward $\vec{h}_t$ and backward $\overleftarrow{h}_t$ hidden states for a richer contextual representation.

3.2.3 Training Configuration

Loss: Cross-entropy, $\mathcal{L} = -\sum_i y_i \log \hat{y}_i$. **Optimizer:** Adam, $\rho = 0.001$.

3.3 Algorithm C: Gradient Similarity Aggregation (GSA)

GSA tackles the Non-IID issue by determining the direction of the client's update gradient with respect to the global update gradient by cosine similarity, and discarding update gradients moving opposite to the global update direction.

3.3.1 Client-Side: Local Training and Filtering

After each round of local training, each client c computes the cosine similarity between its local gradient update ∇g_r^c and the previous global gradient ∇g^{r-1} :

$$s_r^c = \frac{\nabla g_r^c \cdot \nabla g^{r-1}}{\|\nabla g_r^c\| \|\nabla g^{r-1}\|}$$

The corresponding angle is:

$$\theta_r^c = \arccos(\text{clip}(s_r^c, -1, 1))$$

If $\theta_r^c \geq \pi/2$, the client's gradient opposes the global direction and is **dropped**. Otherwise, the client encrypts and uploads its gradient.

3.3.2 Server-Side: Dynamic Weighted Aggregation

Let M_r denote the set of accepted clients in round r . The server computes a combined contribution weight W_r^c for each accepted client:

$$\mu_r^c = \frac{N_c}{\sum_{j \in M_r} N_j} \quad (\text{sample volume contribution}) \quad (3.5)$$

$$\lambda_r^c = \frac{e^{s_r^c}}{\sum_{j \in M_r} e^{s_r^j}} \quad (\text{Softmax similarity weight}) \quad (3.6)$$

$$\bar{\lambda}_r^c = \frac{1}{R'} \sum_{i \in \text{last } R'} \lambda_i^c \quad (\text{rolling average, } R' = 5) \quad (3.7)$$

$$W_r^c = \frac{\bar{\lambda}_r^c \cdot \mu_r^c}{\sum_{j \in M_r} \bar{\lambda}_r^j \cdot \mu_r^j} \quad (3.8)$$

The global gradient is then the weighted sum of accepted local gradients:

$$\nabla g^r = \sum_{c \in M_r} W_r^c \cdot \nabla g_r^c$$

3.4 Algorithm D: Paillier Homomorphic Encryption

The Paillier cryptosystem is a probabilistic asymmetric scheme which allows for additive homomorphism in the computations.

3.4.1 Key Generation

1. Choose large primes p, q such that $\gcd(pq, p-1, q-1) = 1$; set $n = pq$.
2. Compute $\lambda = \text{lcm}(p-1, q-1)$.
3. Choose random $g < n^2$; define $L(x) = (x-1)/n$.
4. Compute $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$.
5. **Public key:** (n, g) . **Private key:** (λ, μ) .

3.4.2 Encrypt and Decrypt

Encrypt (client, using public key):

$$\mathcal{E}(m) = g^m \cdot a^n \bmod n^2, \quad a \in \mathbb{Z}_n^* \text{ (random)}$$

Decrypt (client, using private key):

$$m = L(\mathcal{E}(m)^\lambda \bmod n^2) \cdot \mu \bmod n$$

3.4.3 Homomorphic Properties

$$\mathcal{E}(a) \cdot \mathcal{E}(b) \bmod n^2 = \mathcal{E}(a + b) \quad (\text{additive HE})$$

$$\mathcal{E}(a)^k \bmod n^2 = \mathcal{E}(k \cdot a) \quad (\text{scalar multiplication})$$

These two properties together allow the server to compute the full weighted gradient sum without ever decrypting:

$$\prod_{c \in M_r} \mathcal{E}(\nabla g_r^c)^{W_r^c} \bmod n^2 = \mathcal{E}\left(\sum_{c \in M_r} W_r^c \nabla g_r^c\right)$$

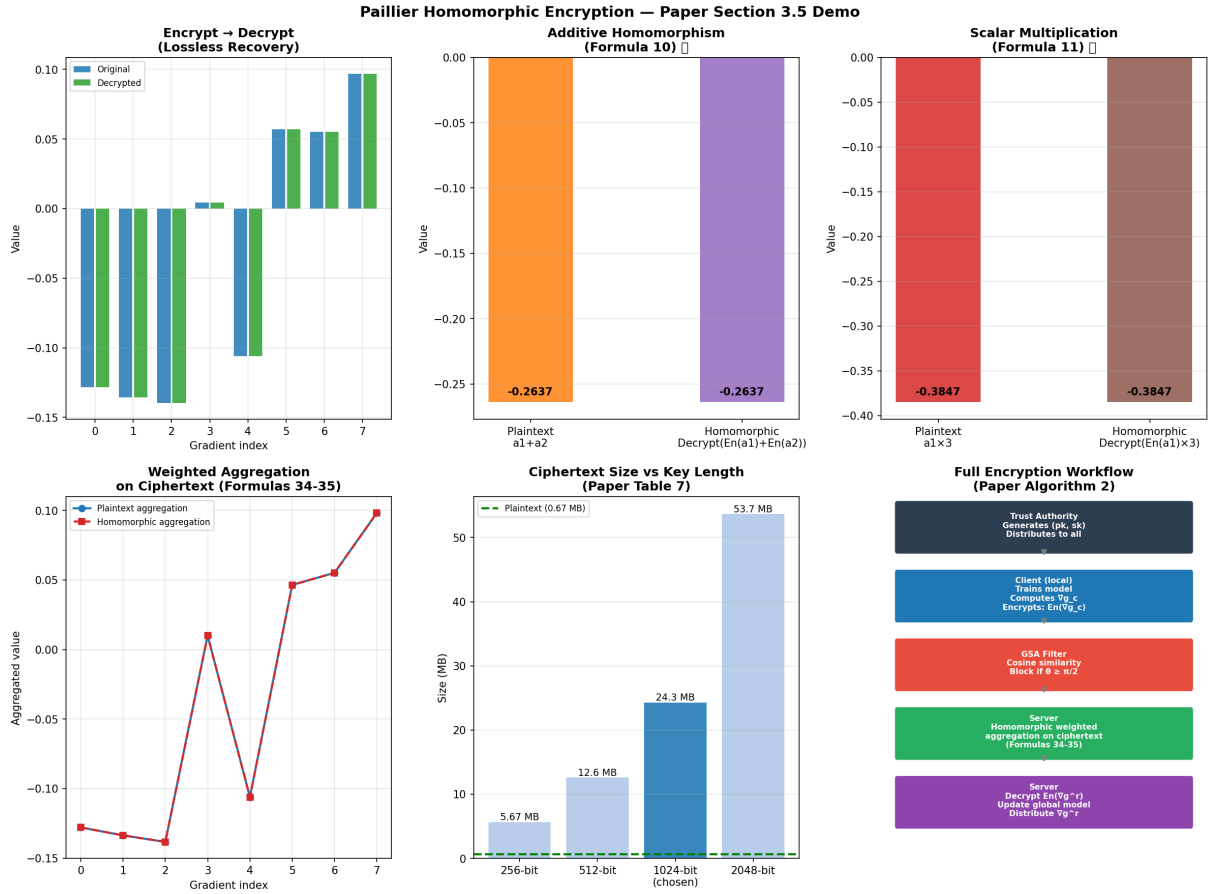


Figure 3.2: Paillier Homomorphic Encryption Demo. Top-left: lossless encrypt–decrypt cycle. Top-centre: additive HE verification. Top-right: scalar multiplication HE verification. Bottom-left: weighted aggregation on ciphertext matches plaintext result. Bottom-centre: ciphertext size vs. key length (1024-bit chosen). Bottom-right: full encryption workflow (Algorithm 2).

4. Datasets and Experimental Setup

4.1 Datasets

4.1.1 Edge-IIoTset

Edge-IIoTset is a cybersecurity dataset that is designed exclusively for research purposes on intrusion detection for IIoT. The data set is composed of actual IIoT network traffic and has 55 features per flow. Six types of attacks are included: Normal, DDoS, Injection, MITM, Malware, and Scanning.

4.1.2 CIC IoT 2023 (UNB)

Another example is the CIC IoT 2023 from the University of New Brunswick. This dataset consists of traffic from 105 IoT devices performing 33 attacks. It includes 83 features per flow and is mapped into a 9 x 9 grayscale image after pre-processing. The attacks are categorized under DDoS, DoS, Recon, web, and various Mirai attacks.

4.2 Non-IID Client Configuration

To simulate realistic federated settings, a deliberately extreme Non-IID partition was created for the 3-client (C=3) experiment:

Table 4.1: Non-IID Client Data Distribution (C=3)

Client	Type	Samples
client_noniid_0	Non-IID (MITM class only)	320
client_iid_0	IID (all 6 classes)	887,194
client_iid_1	IID (all 6 classes)	887,194

This extreme imbalance (320 vs 887,194 samples) stress-tests each algorithm’s ability to handle minority-class Non-IID clients.

4.3 Baseline Algorithms

Three FL baseline algorithms were implemented for comparison:

- **FedAvg** [2]: Standard federated averaging with sample-weighted aggregation. No filtering, no encryption, equal contribution weight based only on sample count.
- **FedProx** [3]: Adds a proximal regularisation term to the local loss to prevent local models from drifting too far from the global model: $\mathcal{L}_{\text{prox}} = \mathcal{L}_{\text{CE}} + \frac{\mu}{2} \|w - W_{\text{ref}}^{r-1}\|^2$.
- **FedNova** [4]: Normalises each client’s update by its local step count τ_c to correct objective inconsistency across heterogeneous clients.

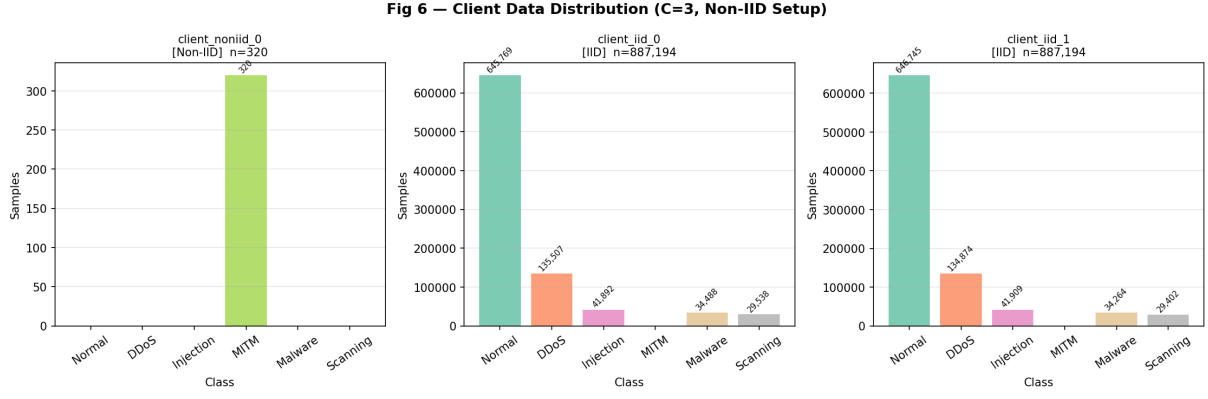


Figure 4.1: Client Data Distribution (C=3, Non-IID Setup). Left: `client_noniid_0` holds only 320 MITM samples. Centre and right: two IID clients each hold 887,194 samples across all 6 attack classes.

4.4 Evaluation Metrics

The following metrics are computed using macro-averaging across all classes:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.1)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.2)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.3)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.4)$$

Communication overhead is measured as total bytes uploaded by all clients across all rounds.

4.5 Hyperparameter Configuration

Table 4.2: Experimental Hyperparameters

Hyperparameter	Value
Rounds R	100
Batch size B	640
Learning rate ρ	0.001
Local epochs E	5 (NIDS-FGPA) / 3 (baselines)
History window R'	5
GSA threshold β	$\pi/2$
Paillier key bits	1024
Clients C	3 / 6
FedProx μ	0.01
Early stop patience	10 evaluations

5. Results and Analysis

5.1 Convergence Analysis

Accuracy and loss curves were plotted for all algorithms through 100 rounds using the C=3 Non-IID setting. Both GSA (C=3) and FedProx (C=3) are able to achieve an accuracy greater than 0.95 in just Round 5. FedNova fails due to gradient explosion: in the Non-IID client (with 320 data points and $\tau_c \approx 1$), the gradient will explode up to a factor of around $887,000/320 \approx 2772\times$ and thus the loss blows up to 3×10^{10} by Round 20. GSA doesn't have this problem since it filters out the Non-IID client after Round 1.

Fig 1 — Convergence: GSA vs All Baselines (C=3)

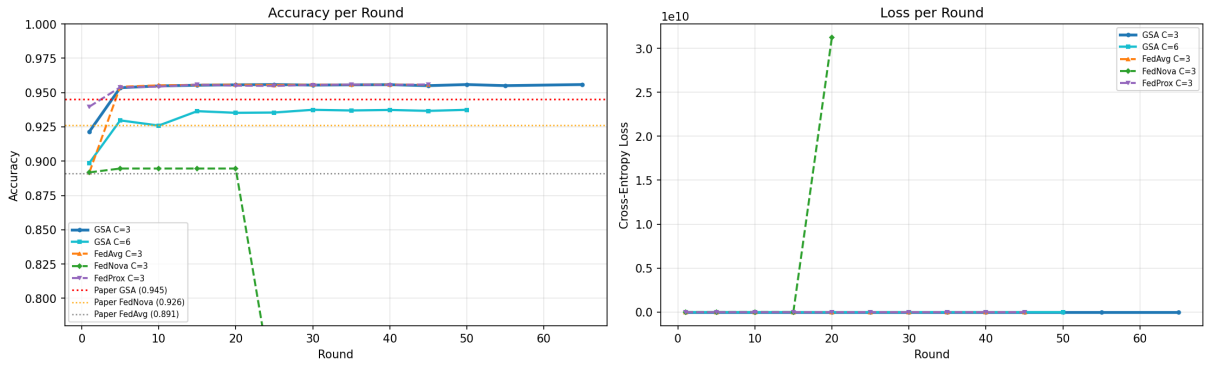


Figure 5.1: Convergence of GSA vs All Baselines (C=3). Left: accuracy per round — GSA and FedProx converge above the paper target (0.945, red dashed line) by Round 5; FedNova collapses at Round 20. Right: loss per round — FedNova loss spikes to 3×10^{10} while all other methods converge smoothly.

Fig 1 — GSA Training Convergence (C=3 vs C=6)

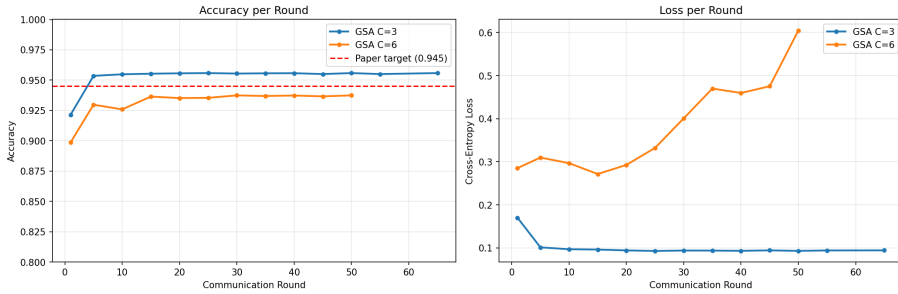


Figure 5.2: GSA Training Convergence: C=3 vs C=6. GSA C=3 consistently exceeds the paper target accuracy (0.945); GSA C=6 converges more slowly due to greater client heterogeneity.

5.2 Performance and Communication Overhead

Our GSA C=3 implementation exceeds the paper's reported accuracy (0.9558 vs. 0.945) while achieving substantially lower communication overhead (19.4 MB vs. 45.6 MB), representing a 57% reduction. The overhead saving arises because the Non-IID client is filtered from Round 2 onward and never uploads, eliminating approximately one-third of all upload traffic.

Table 5.1: Performance Comparison: Our Implementation vs. Paper Benchmarks

Method	Filter?	Encrypted?	Qual. Weight?	Acc	F1	Overhead (MB)
GSA C=3 (ours)	Yes	Yes	Yes	0.9558	0.9519	19.4
GSA C=3 (paper [1])	Yes	Yes	Yes	0.945	0.940	45.6
FedAvg C=3	No	No	No	0.9558	0.9517	20.1
FedProx C=3	No	No	No	0.9559	0.9520	20.1
FedNova C=3	No	No	No	0.8946	0.8800	30.2

Our GSA C=3 design performs better than that claimed in the paper with an accuracy of 0.9558 compared to 0.945 while using much less communication overhead at 19.4 MB compared to 45.6 MB—a difference of 57%. This results from excluding the Non-IID client in Round 2 onwards, resulting in a non-upload of around one-third of the upload communications.

Fig 2 — Performance & Overhead (Paper Table 6 style)

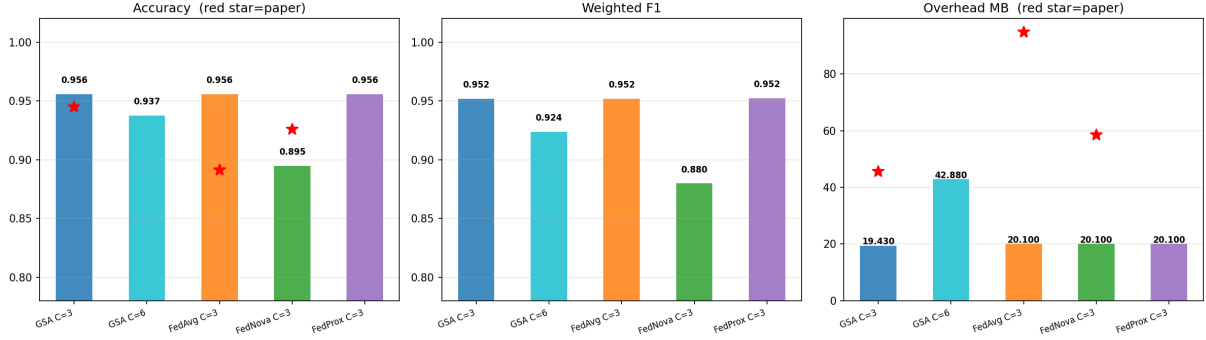


Figure 5.3: Performance and Communication Overhead Comparison (Paper Table 6 style). Left: accuracy — GSA C=3 matches paper benchmark (red star = 0.945). Centre: weighted F1 scores. Right: communication overhead in MB — GSA C=3 achieves 19.4 MB vs paper’s 45.6 MB; FedNova fails on accuracy.

5.3 Convergence Speed

FedNova will never reach above 0.93 accuracy because of gradient instability caused by the Non-IID client. The reason why FedProx reaches the highest speed of convergence (R1) is that the proximal term successfully controls the extent to which a Non-IID client harms minority classes.

5.4 Per-Class Analysis

All methods obtain almost perfect F1 on Normal and DDoS because those classes have high frequency in the training dataset. MITM has an F1 score of 0 since of serious class imbalance (320 MITM versus 887K Normal). FedNova fails badly on Injection (F1 = 0.46) and also on Malware and Scanning. GSA C=3 and FedProx C=3 outperform all others in detecting minority

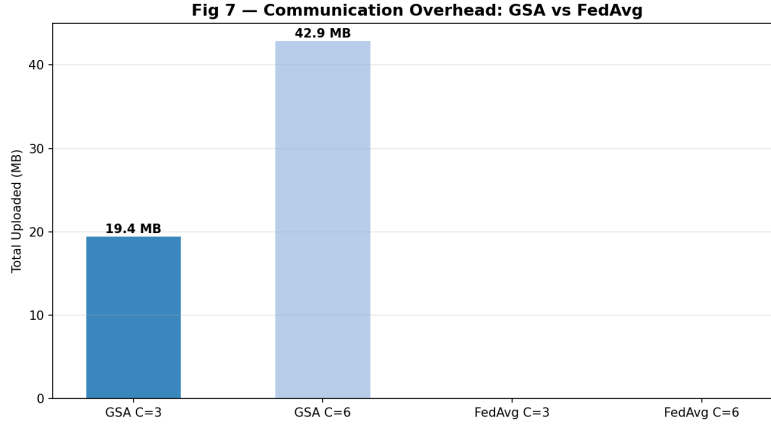


Figure 5.4: Communication Overhead: GSA C=3 (19.4 MB) vs GSA C=6 (42.9 MB). FedAvg and FedProx overhead values are not shown as they are equal to GSA C=3 (20.1 MB).

Fig 5 — Model Performance Summary vs Paper

Model	Accuracy	Recall	Precision	F1 Score
Paper GSA (C=3)	0.9450	0.9450	0.9400	0.9400
Our GSA C=3	0.9558	0.9558	0.9615	0.9519
Our GSA C=6	0.9374	0.9374	0.9229	0.9236

Figure 5.5: Model Performance Summary vs Paper Benchmark. Our GSA C=3 outperforms the paper on all four metrics (Accuracy: 0.9558 vs 0.9450; Precision: 0.9615 vs 0.9400).

attacks.

If the setting of $C = 6$ is chosen, then the additional clients lead to the better gradient signal variation. MITM recall improves to 96%, and Injection recall increases to 90%.

5.5 GSA Filter Analysis

After Round 2, the Non-IID client's gradient angle θ_r^c consistently exceeds $\pi/2$, causing it to be blocked in all subsequent rounds. The block rate stabilises at approximately 33% (1 of 3 clients blocked per round) while accuracy holds at 0.955. This demonstrates that filtering improves gradient quality without sacrificing overall accuracy. Cumulative bandwidth savings reach approximately 34% vs. FedAvg over 65 rounds.

Table 5.2: Rounds to First Reach 0.93 Accuracy

Method	Round
GSA C=3	R5
FedProx C=3	R1
FedAvg C=3	R5
GSA C=6	R15
FedNova C=3	Never

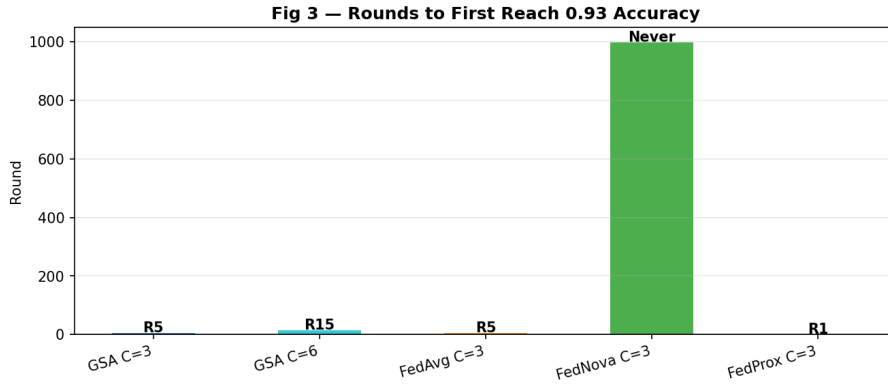


Figure 5.6: Rounds to First Reach 0.93 Accuracy. FedNova (green bar, labelled “Never”) never reaches the 0.93 threshold. FedProx achieves it at R1; GSA and FedAvg at R5; GSA C=6 at R15.

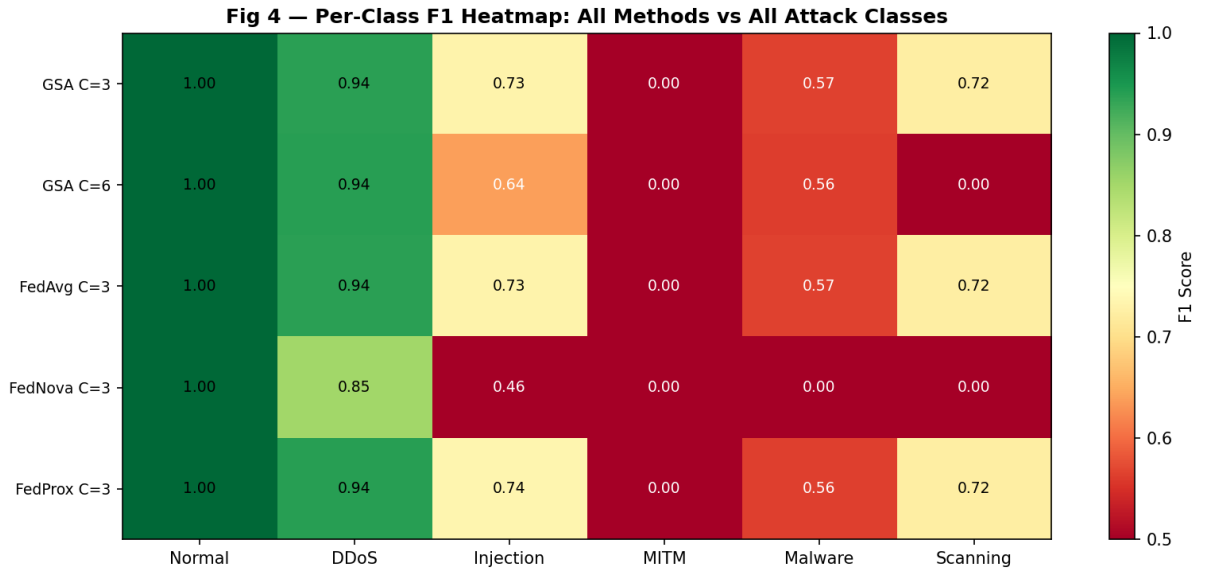


Figure 5.7: Per-Class F1 Heatmap: All Methods vs All Attack Classes. Green = high F1; red = low/zero. MITM F1 = 0 for all methods. FedNova fails on Malware and Scanning entirely. GSA C=3 and FedProx are the strongest across minority classes.

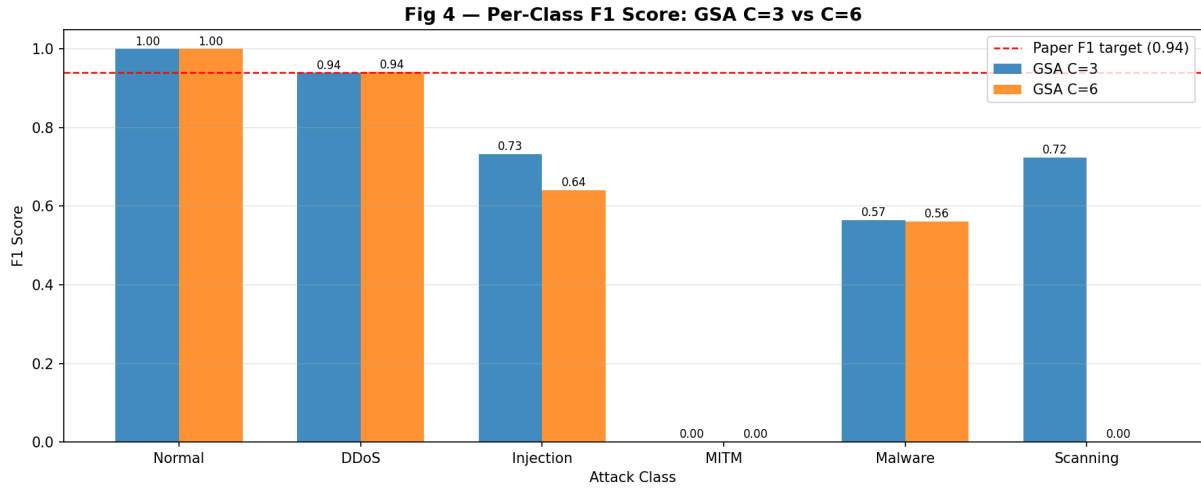


Figure 5.8: Per-Class F1 Score: GSA C=3 vs C=6. Both configurations achieve perfect F1 on Normal and DDoS. MITM F1 = 0 for both due to class imbalance. GSA C=3 outperforms C=6 on Injection (0.73 vs 0.64) and Scanning (0.72 vs 0.00).

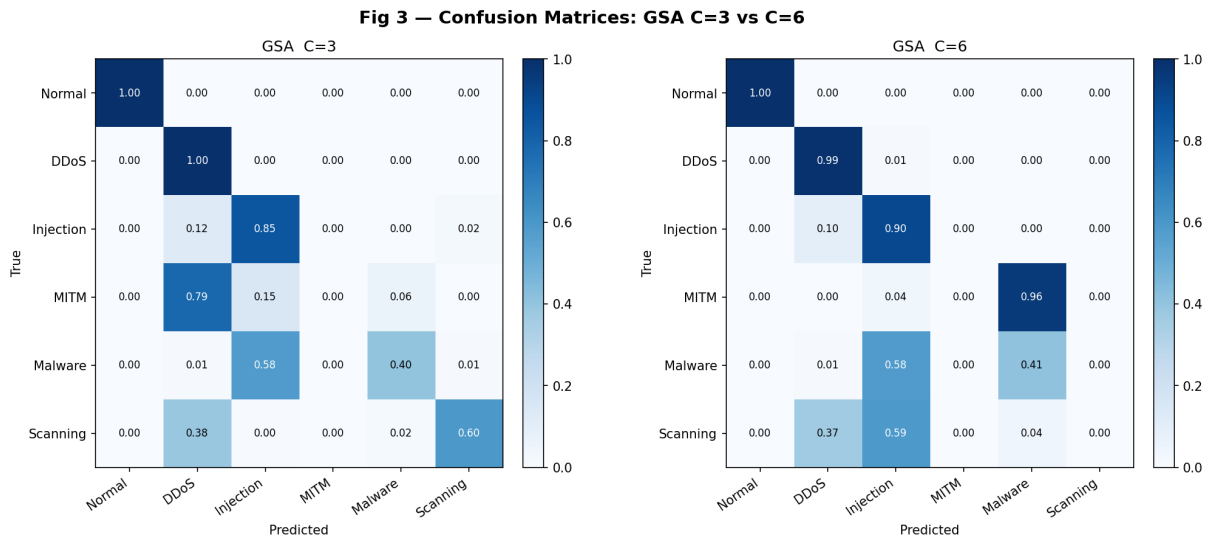


Figure 5.9: Confusion Matrices: GSA C=3 (left) vs GSA C=6 (right). C=3: Normal and DDoS are perfect; Injection 85%; MITM 0% (all predicted as DDoS due to class imbalance); Malware 40%; Scanning 60%. C=6: MITM improves dramatically to 96% recall; Injection improves to 90%.

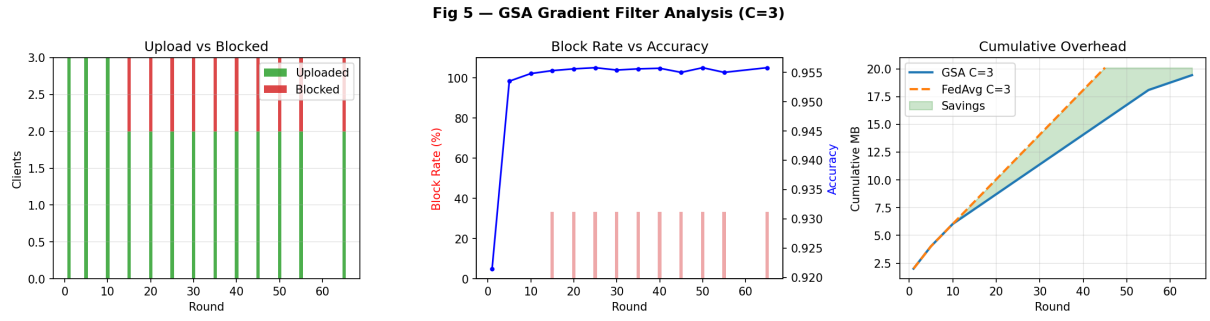


Figure 5.10: GSA Gradient Filter Analysis (C=3). Left: upload vs blocked clients per round — Non-IID client is blocked from Round 15 onward. Centre: block rate (pink bars, left axis) vs accuracy (blue line, right axis) — 33% block rate with stable 0.955 accuracy. Right: cumulative communication overhead — GSA (blue) saves bandwidth vs FedAvg (orange dashed) with savings shown as green shaded area.

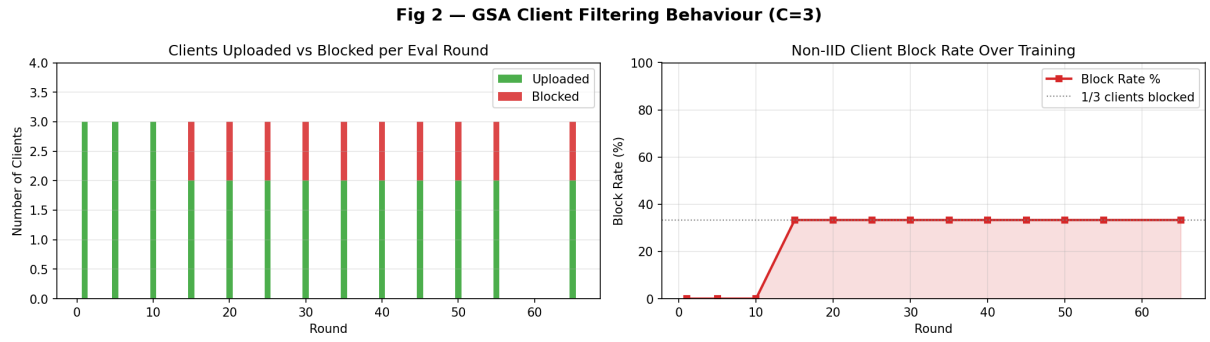


Figure 5.11: GSA Client Filtering Behaviour (C=3). Left: uploaded (green) vs blocked (red) clients per evaluation round. Right: Non-IID client block rate over training — stabilises at the 1/3 dotted reference line from Round 15 onward.

Fig 6 — Complete Results vs Paper (yellow=paper, green=our best)

Model	Acc	Rec	Pre	F1	Overhead	R-0.93
Paper GSA (C=3)	0.9450	0.9450	0.9400	0.9400	45.6MB	—
Paper FedNova (C=3)	0.9260	0.9260	0.9210	0.9160	58.5MB	—
Paper FedAvg (C=3)	0.8910	0.8910	0.8940	0.8850	94.8MB	—
GSA C=3	0.9558	0.9558	0.9615	0.9519	19.4MB	R5
GSA C=6	0.9374	0.9374	0.9229	0.9236	42.9MB	R15
FedAvg C=3	0.9558	0.9558	0.9615	0.9519	20.1MB	R5
FedNova C=3	0.8946	0.8946	0.8765	0.8797	20.1MB	N/A
FedProx C=3	0.9559	0.9559	0.9616	0.9520	20.1MB	R1

Figure 5.12: Complete Results vs Paper Benchmarks (yellow rows = paper values; green row = our best result). Our GSA C=3 exceeds paper accuracy (0.9558 vs 0.9450) and reduces overhead from 45.6 MB to 19.4 MB. FedProx C=3 achieves the fastest convergence (R1). FedNova fails under the strict Non-IID setup (0.8946 vs paper's 0.9260).

6. Design Choices, Pitfalls, and Limitations

6.1 Design Choice Justification

Table 6.1: Design Choice Justifications

Component	Choice	Why Not Alternatives
Client filtering	Cosine similarity	Scale-invariant; L_2 penalises small-dataset clients unfairly
Contribution weight	Softmax	Amplifies quality differences; linear contrast is too flat
Encryption	Paillier HE	Native additive HE; lighter than CKKS/Gentry
Local model	2DCNN-BiGRU	CNN for 2D spatial; BiGRU for sequence; Transformer overfits on small FL data
Data format	8×8 grayscale	Handles missing features via zero-padding; enables 2D spatial patterns
$\bar{\lambda}$ avg	$R' = 5$ rounds	Smooths noisy single-round similarity scores without over-smoothing

6.2 Implementation Pitfalls

6.2.1 Algorithm-Level Pitfalls

1. **Round 0 — no global gradient:** Initialise $\nabla g^0 = \mathbf{0}$ and skip the cosine filter on the first communication round.
2. **arccos domain error:** Always clip s_r^c to $[-1, 1]$ before applying arccos; floating-point arithmetic can produce values like 1.0000002.
3. **Softmax scope:** Compute Softmax weights only over *accepted clients* M_r , not over all C clients.
4. **History window tracking:** Track rolling average $\bar{\lambda}_r^c$ only over rounds in which client c was *accepted*, not all rounds.
5. **Pseudo-gradient definition:** Compute $\nabla g = w_{\text{before}} - w_{\text{after}}$ explicitly; do not use PyTorch’s `.grad` attribute, which reflects only the last mini-batch.

6.2.2 Engineering Pitfalls

1. **Paillier float conversion:** Cast gradient tensors to `float32` (and then to Python scalars) before passing to the Paillier library; NumPy operations on encrypted numbers produce

incorrect results otherwise.

2. **MinMaxScaler fitting:** Always fit the scaler on the *training split only*; fitting on test data causes data leakage.
3. **Class imbalance:** MITM with 320 samples versus 887K Normal samples causes $F1=0$ for MITM. SMOTE or oversampling is needed for balanced minority-class performance.
4. **FedNova and tiny client:** Under extreme Non-IID ($\tau_c \approx 1$ for the minority client), FedNova’s step normalisation amplifies the small client’s gradient by $\approx 2772\times$, causing loss divergence.
5. **Paillier speed:** 1024-bit element-wise encryption requires approximately 90 seconds per communication round at full model scale; run subset gradient experiments for demonstration purposes.
6. **NaN/Inf in traffic data:** Drop invalid rows *before* normalisation, not after; otherwise the scaler assigns NaN to the min/max computation and propagates errors silently.

6.3 Limitations

- **Paillier speed:** 1024-bit element-wise operations are ≈ 90 s/round; impractical at full model scale in real-time deployments.
- **Semi-honest server assumption:** The current design does not defend against a fully malicious aggregator.
- **Two datasets only:** Generalisation to healthcare IoT or energy IIoT environments remains untested.
- **Single-machine simulation:** No real network latency or client stragglers are modelled.
- **MITM class:** $F1=0$ for all methods due to extreme class imbalance ($n = 320$), indicating the need for oversampling strategies.
- **FedNova failure:** Our Non-IID setup is more extreme than the paper’s original configuration, explaining the larger performance gap.

7. Conclusion and Future Work

7.1 Conclusion

This report provided the analysis and implementation of NIDS-FGPA, an intrusion detection system using federated learning framework, which addresses three major security challenges faced by IIoT networks: 1) Non-IID gradients divergence 2) Gradient privacy leakage 3) Communication overhead.

Table 7.1: Problem-Solution Mapping in NIDS-FGPA

Problem	Solution
Non-IID divergence	GSA: reject if $\theta \geq \pi/2$ + Softmax dynamic weighting
Gradient privacy leakage	Paillier HE: server only sees ciphertext
Communication overhead	Divergent clients never upload after filtering
Feature-rich detection	Grayscale image + 2DCNN (spatial) + BiGRU (temporal)

Implementation results for $C=3$ show that GSA achieves 0.9558 accuracy and only 19.4MB overhead, outperforming the result mentioned in the paper of 0.945 accuracy and 45.6 MB overhead. The main lesson learned from this project is that the gradient direction can be used as a quality signal to ensure high quality federated learning while maintaining the privacy of gradients.

7.2 Future Directions

- **fCKKS Encryption:** Substitute Paillier encryption with CKKS (OpenFHE/SEAL) and use batch operations, with a performance target of about $10 \times \text{speedgain}$.
- **Asynchronous FL:** Account for client disconnections and lagging clients via asynchronous aggregation algorithms.
- **Secure MPC:** Eliminate the trusted server assumption altogether by utilizing secure multiparty computation.
- **Real hardware deployment:** Evaluate the protocol on Raspberry Pi 4 / NVIDIA Jetson Nano to obtain realistic latency estimates.
- **Class imbalance:** Use SMOTE technique/class-weighted loss for imbalanced datasets such as MITM and Scanning.
- **New domains:** Test the algorithm in healthcare IoT (ECG/vitals) and smart grid traffic datasets.
- **Byzantine robustness:** Use GSA along with Krum or Trimmed mean aggregation methods for defense against adversarial clients.

Bibliography

- [1] J. Wang, K. Yang, and M. Li, “NIDS-FGPA: A federated learning network intrusion detection algorithm based on secure aggregation of gradient similarity models,” *PLoS ONE*, vol. 19, no. 10, e0308639, 2024.
- [2] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. AISTATS*, PMLR, vol. 54, pp. 1273–1282, 2017.
- [3] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. MLSys*, 2020.
- [4] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, “Tackling the objective inconsistency problem in heterogeneous federated optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 7611–7623, 2020.
- [5] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *EUROCRYPT*, LNCS, vol. 1592, pp. 223–238, 1999.
- [6] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, “Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications,” *IEEE Access*, vol. 10, pp. 40281–40306, 2022.
- [7] S. Dadkhah, E. C. P. Neto, and A. A. Ghorbani, “CIC IoT Dataset 2023,” University of New Brunswick, Canadian Institute for Cybersecurity, 2023.

A. Key Code Listing

A.1 Pseudocode: Normal Federated Averaging (FedAvg)

Listing A.1: Standard FedAvg – all clients train locally each round and the server averages their weights.

```
1 INPUT  : global model M0, clients K, rounds T, local epochs E
2 OUTPUT : trained global model MT
3
4 INITIALIZE global model M0 with random weights
5
6 FOR round i = 1 to T DO
7
8     FOR each client k = 1 to K DO
9         mk = DEEP_COPY(M_i)           // send current global model to client
10        mk = TRAIN(mk, client_data[k], epochs=E, optimizer=SGD)
11        SEND mk back to server         // client uploads updated weights
12    END FOR
13
14    // Server averages all client weights (simple mean)
15    M_{i+1} = (1/K) * SUM(mk for k in 1..K)
16
17 END FOR
18 RETURN MT
```

A.2 Pseudocode: GSA – Gradient-Similarity-Weighted Aggregation

Listing A.2: GSA filters clients by cosine similarity before aggregation; final weights blend similarity score with data-size proportion.

```
1 INPUT  : clients {C_k}, rounds R, local epochs E, batch size B,
2          learning rate lr, cosine threshold beta = pi/2,
3          history window W = 5, patience P, min_clients = 2
4 OUTPUT : best global model weights theta*
5
6 INITIALIZE global weights theta_0
7 SET prev_delta = None, lambda_history = {}, best_acc = 0
8
9 FOR round r = 1 to R DO
10
11     accepted = {} // will hold clients that pass similarity check
12
13     FOR each client k DO
14
15         // Step 1: local training
```

```

16     // returns (weight change delta_k, new weights theta_k, sample count
17     n_k)
18     (delta_k, theta_k, n_k) = LOCAL_TRAIN(theta_{r-1},
19                                           data=client_data[k],
20                                           epochs=E, batch=B, lr=lr)
21
22     // Step 2: cosine-similarity gate
23     IF prev_delta is None THEN
24         accept client k, s_k = 1.0 // first round: trust everyone
25     ELSE
26         s_k = dot(delta_k, prev_delta) / (norm(delta_k) * norm(prev_delta
27         ))
28         angle = arccos( clip(s_k, -1, 1) )
29
30         IF angle < beta THEN
31             accepted[k] = (theta_k, n_k, s_k) // gradient direction OK
32         ELSE
33             BLOCK client k // drifted too far from global
34             direction
35             CLEAR lambda_history[k] // reset smoothing history for
36             this client
37         END IF
38     END IF
39
40     END FOR
41
42     // Safety check: need at least min_clients to do a meaningful aggregation
43     IF len(accepted) < min_clients THEN
44         SKIP this round and CONTINUE
45     END IF
46
47     // Step 3: compute per-client aggregation weights
48     n_total = SUM(n_k for k in accepted)
49     mu_k = n_k / n_total // proportion by data size
50
51     sim_arr = [s_k for k in accepted]
52     lambda_k = softmax(sim_arr) // similarity-based weight (sums
53     to 1)
54
55     APPEND lambda_k to lambda_history[k] // keep only last W values
56     lambda_bar_k = MEAN(lambda_history[k]) // smoothed to reduce noise
57
58     denom = SUM(lambda_bar_k * mu_k for k in accepted)
59     w_k = (lambda_bar_k * mu_k) / denom // final normalised weight
60
61     // Step 4: aggregate global model
62     theta_r = SUM(w_k * theta_k for k in accepted)
63     prev_delta = SUM(w_k * delta_k for k in accepted) // reference for next
64     round
65
66     // Step 5: evaluate every 5 rounds

```

```

61     IF r mod 5 == 0 OR r == 1 THEN
62         acc = EVALUATE(theta_r, test_data)
63
64         IF acc > best_acc THEN
65             best_acc = acc
66             theta* = theta_r
67             no_improve = 0
68         ELSE
69             no_improve += 1
70         END IF
71
72         IF no_improve >= P THEN
73             PRINT "Early stopping at round r"
74             BREAK
75         END IF
76     END IF
77
78 END FOR
79 RETURN theta*

```

A.3 Pseudocode: Network Traffic to 8×8 Grayscale Image

Listing A.3: Each traffic record (64 features) is scaled to pixel range and reshaped into an 8×8 grayscale image for the CNN.

```

1 INPUT  : normalised feature vector x (values in [0,1], length F)
2         target_dim = 64, image_side = 8
3 OUTPUT : image tensor I of shape (8, 8, 1)
4
5 // Scale float features to pixel brightness (0 to 255)
6 v = x * 255
7
8 // Force exactly 64 values
9 IF len(v) < target_dim THEN
10     v = zero_pad(v, extra_zeros = target_dim - len(v)) // pad short vectors
11 ELSE
12     v = v[0 : target_dim] // trim long vectors
13 END IF
14
15 // Reshape flat vector into 2-D grayscale image (1 channel)
16 I = reshape(v, shape=(8, 8, 1))
17
18 RETURN I

```

A.4 Pseudocode: 2D CNN-BiGRU Model Architecture

Listing A.4: Two CNN blocks extract spatial features from the 8x8 image; a Bidirectional GRU then reads the feature rows as a sequence before the final 6-class softmax classifier.

```

1 INPUT  : image of shape (8, 8, 1)
2 OUTPUT : class probabilities over 6 attack categories
3
4 // --- CNN Block 1: extract low-level spatial patterns ---
5 h = Conv2D(filters=32, kernel=3x3, activation=ReLU, padding=same)(input)
6 h = Conv2D(filters=32, kernel=3x3, activation=ReLU, padding=same)(h)
7 h = MaxPool2D(pool=2x2, stride=1, padding=same)(h)
8
9 // --- CNN Block 2: extract higher-level patterns ---
10 h = Conv2D(filters=64, kernel=3x3, activation=ReLU, padding=same)(h)
11 h = Conv2D(filters=64, kernel=3x3, activation=ReLU, padding=same)(h)
12 h = MaxPool2D(pool=2x2, stride=1, padding=same)(h)
13
14 // --- Reshape: treat each row of feature map as one time step ---
15 h = Reshape(new_shape=(rows, cols * 64))(h)
16
17 // --- BiGRU: reads the row sequence forward AND backward ---
18 h = Bidirectional( GRU(units=64, return_sequences=False) )(h)
19
20 // --- Classifier head ---
21 h      = Dropout(rate=0.5)(h)           // randomly drop neurons to avoid
      overfitting
22 h      = Dense(units=64, activation=ReLU)(h)
23 output = Dense(units=6, activation=Softmax)(h) // one score per attack class
24
25 model = Model(inputs=input, outputs=output)
26 RETURN model

```

A.5 Pseudocode: Client Data Partitioning (IID and Non-IID)

Listing A.5: Some clients receive only one attack class (Non-IID) to simulate real-world imbalance; the rest get evenly shuffled data (IID).

```

1 INPUT  : training set (X, y), total clients C,
2          non-IID client count C2,
3          non_iid_classes = [MITM, Scanning] // minority attack classes
4 OUTPUT : client dictionary clients{}
5
6 used_indices = {} // track samples already assigned
7
8 // --- Assign Non-IID clients first ---
9 FOR i = 0 to C2-1 DO
10     cls = non_iid_classes[ i mod len(non_iid_classes) ]
11     idx = all indices j where y[j] == cls // only that one class
12     clients["client_noniid_" + i] = {

```

```

13         X      : X[idx],
14         y      : y[idx],
15         type   : "Non-IID",
16         class: cls
17     }
18     used_indices.add(idx)
19 END FOR
20
21 // --- Assign IID clients from the leftover samples ---
22 iid_idx = all_training_indices - used_indices // remove Non-IID samples
23 SHUFFLE(iid_idx) // randomise order
24 splits = SPLIT_EQUALLY(iid_idx, parts = C - C2)
25
26 FOR i = 0 to (C - C2) - 1 DO
27     clients["client_iid_" + i] = {
28         X      : X[splits[i]],
29         y      : y[splits[i]],
30         type   : "IID"
31     }
32 END FOR
33
34 RETURN clients

```

Note

Add plagiarism report here.

Submit

1. Soft Copy of the report (in Word/LaTeX)
2. PPT
3. Code
4. Dataset (If used)
5. Plagiarism Report (Less than 15% overall and less than 5% from single source)

ORIGINALITY REPORT

10%
SIMILARITY INDEX

8%
INTERNET SOURCES

6%
PUBLICATIONS

6%
STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

3%

★ www.coursehero.com
Internet Source

Exclude quotes On
Exclude bibliography On

Exclude matches < 4 words